

Algorithmen Übungen

HINWEIS ZU DEN UNIT TESTS

Für alle deine Unit Tests gilt:

- 100 % Testabdeckung
- AAA-Pattern
- Mindestens einen zusätzlichen Edge Case
(d. h. du hast dann theoretisch mehr als 100 % Testabdeckung)

Hello World

Schreibe eine main-Methode, die "Hello World" in der Konsole ausgibt.



Hello World: Lösung

```
class HelloWorld {  
  
    public static void main(String[] args) {  
        System.out.println("Hello World");  
    }  
}
```

Hello World – Unit Test

Schreibe deine main-Methode aus der vorherigen Aufgabe "Hello World" um.

Entferne `System.out` aus der Methode und füge stattdessen ein `"return"` hinzu, das `"Hello World"` zurückgibt, um die Methode testen zu können (anschließend ist es keine main-Methode mehr).

Schreibe einen Unit Test für deine überarbeitete Methode.

Hello World: Lösung

```
class HelloWorldTest {  
  
    @Test  
    void testHelloWorld() {  
        // Arrange  
        var expected = "Hello World";  
        // Act  
        var actual = HelloWorld.helloWorld();  
        // Assert  
        assertEquals(expected, actual);  
    }  
}
```

FizzBuzz

Für Zahlen von 1 bis 100, gebe für jede Zahl die Zahl selbst aus,

- außer wenn die Zahl durch 3 teilbar ist, gebe "Fizz" aus,
- wenn sie durch 5 teilbar ist, gebe "Buzz" aus,
- und wenn sie durch beide Zahlen teilbar ist, gebe "FizzBuzz" aus.

Die Ausgabe soll in der Konsole erfolgen.

Schreibe das Programm in einer main-Methode.

FizzBuzz: Lösung

```
public static void main(String[] args) {  
    for (int i = 1; i <= 100; i++) {  
        if (i % 3 == 0 && i % 5 == 0) {  
            System.out.println("FizzBuzz");  
        } else if (i % 3 == 0) {  
            System.out.println("Fizz");  
        } else if (i % 5 == 0) {  
            System.out.println("Buzz");  
        } else {  
            System.out.println(i);  
        }  
    }  
}
```


FizzBuzz – Unit Test

Schreibe deine Lösung "FizzBuzz" so um, dass sie (Unit Test) testbar ist.

Schreibe deine Methode außerdem so um, dass man über Parameter den Zahlenbereich ändern kann, d. h. den Startwert und Endwert für die FizzBuzz-Schleife.

Schreibe Unit Tests für deine überarbeitete Version (orientiere dich hierzu an Hello World).

FizzBuzz – Unit Test: Lösung

```
class FizzBuzzV2 {  
  
    static List<String> generate(int number) {  
        return generate(number, number);  
    }  
  
    static List<String> generate(int start, int end) {  
        List<String> result = new ArrayList<>();  
        for (int i = start; i <= end; i++) {  
            result.add(getFizzBuzzFor(i));  
        }  
        return result;  
    }  
  
    private static String getFizzBuzzFor(int i) {  
        if (i % 3 == 0 && i % 5 == 0) {  
            return "FizzBuzz";  
        } else if (i % 3 == 0) {  
            return "Fizz";  
        } else if (i % 5 == 0) {  
            return "Buzz";  
        } else {  
            return String.valueOf(i);  
        }  
    }  
}
```

```
@Test  
void fizz(){  
    //Arrange  
    var expected = List.of("Fizz");  
    //Act  
    var actual = FizzBuzzV2.generate(3);  
    //Assert  
    assertEquals(expected, actual);  
}
```

```
@Test  
void buzz(){  
    //Arrange  
    var expected = List.of("Buzz");  
    //Act  
    var actual = FizzBuzzV2.generate(5);  
    //Assert  
    assertEquals(expected, actual);  
}
```

```
@Test  
void fizzBuzz(){  
    //Arrange  
    var expected = List.of("FizzBuzz");  
    //Act  
    var actual = FizzBuzzV2.generate(15);  
    //Assert  
    assertEquals(expected, actual);  
}
```

```
@Test  
void number(){  
    //Arrange  
    var expected = List.of("1");  
    //Act  
    var actual = FizzBuzzV2.generate(1);  
    //Assert  
    assertEquals(expected, actual);  
}
```

```
@Test  
void multipleValues() {  
    // Arrange  
    List<String> expected = List.of("1", "2", "Fizz", "4", "Buzz", "Fizz", "7", "8", "Fizz", "Buzz", "11", "Fizz", "13", "14", "FizzBuzz");  
    // Act  
    var result = FizzBuzzV2.generate(1, 15);  
    // Assert  
    assertEquals(expected, result);  
}
```

Object Caster

Schreibe eine Methode, die eine Liste mit Objekten erhält. D.h., die Liste kann Objekte vom beliebigen Typ enthalten, z. B. Integer, Double, und String.

Iteriere über die Liste und berechne **für jede Ganzzahl (Integer) das Quadrat**. Gebe die Quadrate als neue Liste zurück:

```
static List<Integer> toSquaredIntegerList(List<Object> objects)
```

Falls ein Objekt nicht gecastet werden kann, soll das Objekt **nicht** in die Ergebnisliste mit aufgenommen werden (es wird ignoriert).

Schreibe einen Unit Test für die Klasse.

Bonusaufgabe:

Schreibe deine Lösung so um, dass sie beliebige Zahlen (Integer, Byte, Short, Long, Float, Double) umwandelt und quadriert. Der Rückgabetyt bleibt hierbei List<Integer>.

Tipp: java.lang.Number

Object Caster: Lösung

```
class ObjectCaster {  
  
    static List<Integer> toSquaredIntegerList(List<Object> objects) {  
        List<Integer> result = new ArrayList<>(objects.size());  
  
        for (Object object : objects) {  
            if (object instanceof Integer) {  
                Integer integer = (Integer) object;  
                result.add(integer * integer);  
            }  
        }  
  
        return result;  
    }  
  
    // Alternative als Stream:  
    static List<Integer> toSquaredIntegerListWithStream(List<Object> objects) {  
        return objects.stream()  
            .filter(object -> object instanceof Integer)  
            .map(object -> (Integer) object)  
            .map(number -> number * number)  
            .toList();  
    }  
}
```

Object Caster: Lösung

```
@Test
void negativeNumbers() {
    // Arrange
    List<Object> objectList = List.of(1, -3);

    // Act
    List<Integer> integerList = ObjectCaster.toSquaredIntegerList(objectList);

    // Assert
    assertEquals(List.of(1, 9), integerList);
}
```

```
@Test
void mixedTypes() {
    // Arrange
    List<Object> objectList = List.of(1, "test", 2, 3.5, 3);

    // Act
    List<Integer> integerList = ObjectCaster.toSquaredIntegerList(objectList);

    // Assert
    assertEquals(List.of(1, 4, 9), integerList);
}
```

```
@Test
void onlyIntegers() {
    // Arrange
    List<Object> objectList = List.of(2, 3, 4);

    // Act
    List<Integer> integerList = ObjectCaster.toSquaredIntegerList(objectList);

    // Assert
    assertEquals(List.of(4, 9, 16), integerList);
}
```

```
@Test
void emptyList() {
    // Arrange
    List<Object> objectList = Collections.emptyList();

    // Act
    List<Integer> integerList = ObjectCaster.toSquaredIntegerList(objectList);

    // Assert
    assertTrue(integerList.isEmpty());
}
```

```
@Test
void noIntegers() {
    // Arrange
    List<Object> objectList = List.of("a", "b", 3.5, true);

    // Act
    List<Integer> integerList = ObjectCaster.toSquaredIntegerList(objectList);

    // Assert
    assertTrue(integerList.isEmpty());
}
```

```
@Test
void withNull() {
    // Arrange
    List<Object> objectList = new ArrayList<Object>();
    objectList.add(2);
    objectList.add(null);
    objectList.add(3);

    // Act
    List<Integer> integerList = ObjectCaster.toSquaredIntegerList(objectList);

    // Assert
    assertEquals(List.of(4, 9), integerList);
}
```

Object Caster: Lösung (Bonusaufgabe)

```
class ObjectCaster {  
  
    static List<Integer> toSquaredIntegerList(List<Object> objects) {  
        List<Integer> result = new ArrayList<>(objects.size());  
  
        for (Object object : objects) {  
            if (object instanceof Number) {  
                Number number = (Number) object;  
                // Wandle zunächst alles in double um,  
                // damit vorhandene Nachkommastellen noch mitberechnet werden.  
                double doubleValue = number.doubleValue();  
                double squared = doubleValue * doubleValue;  
                // Wandle berechnete Werte anschließend um in Integer.  
                result.add((int)squared);  
            }  
        }  
  
        return result;  
    }  
}
```

```
// Alternative mit Pattern Matching  
static List<Integer> toSquaredIntegerListPatternMatching(List<Object> objects) {  
    List<Integer> result = new ArrayList<>(objects.size());  
  
    for (Object object : objects) {  
        if (object instanceof Number number) {  
            double doubleValue = number.doubleValue();  
            double squared = doubleValue * doubleValue;  
            result.add((int)squared);  
        }  
    }  
  
    return result;  
}
```

```
// Alternative als Stream  
static List<Integer> toSquaredIntegerList(List<Object> objects) {  
    return objects.stream()  
        .filter(object -> object instanceof Number)  
        .map(object -> ((Number) object))  
        .mapToDouble(object -> object.doubleValue())  
        .map(doubleValue -> doubleValue * doubleValue)  
        .mapToInt(doubleValue -> (int)doubleValue)  
        .boxed()  
        .toList();  
}
```

ListConverter

Schreibe eine Klasse ListConverter und implementiere die folgenden Methoden:

// Wandle die gegebene List<Integer> in ein Integer[]-Array um und gebe das Array zurück.

Integer[] **toArray**(List<Integer> list)

// Wandle das gegebene Integer[]-Array in eine List<Integer> um und gebe die List zurück.

List<Integer> **toList**(Integer[] array)

Schreibe einen Unit Test die Klasse.

ListConverter: Lösung

```
class ListConverter {

    Integer[] toArray(List<Integer> list) {
        return list.toArray(Integer[]::new);

// return list.toArray(new Integer[0]);

// Integer[] array = new Integer[list.size()];
// for (int i = 0; i < list.size(); i++) {
//     array[i] = list.get(i);
// }
// return array;
    }

    List<Integer> toList(Integer[] array) {
        return Arrays.asList(array);

// return List.of(array); // erzeugt eine immutable Liste

// var list = new ArrayList<Integer>(array.length);
// list.addAll(List.of(array));
// return list;

// for (Integer integer : array) {
//     list.add(integer);
// }

// return List.of(array);
    }
}
```

```
class ListConverterTest {
```

```
@Test
void testToListWithNonEmptyArray() {
    // Arrange
    var array = new Integer[]{1, 2, 3};
    var listConverter = new ListConverter();

    // Act
    var result = listConverter.toList(array);

    // Assert
    assertEquals(Arrays.asList(1, 2, 3), result);
}
```

```
@Test
void testToListWithEmptyArray() {
    // Arrange
    var array = new Integer[]{};
    var listConverter = new ListConverter();

    // Act
    var result = listConverter.toList(array);

    // Assert
    assertTrue(result.isEmpty());
}
```

```
@Test
void testToListWithNull() {
    // Arrange
    var array = new Integer[]{1, null, 3};
    var listConverter = new ListConverter();

    // Act
    var result = listConverter.toList(array);

    // Assert
    assertEquals(Arrays.asList(1, null, 3), result);
}
```

```
@Test
void testToArrayWithNonEmptyList() {
    // Arrange
    var list = List.of(1, 2, 3);
    var listConverter = new ListConverter();

    // Act
    var result = listConverter.toArray(list);

    // Assert
    assertEquals(new Integer[]{1, 2, 3}, result);
}
```

```
@Test
void testToArrayWithEmptyList() {
    // Arrange
    List<Integer> list = new ArrayList<>();
    var listConverter = new ListConverter();

    // Act
    var result = listConverter.toArray(list);

    // Assert
    assertEquals(new Integer[] {}, result);
}
```

```
@Test
void testToArrayWithNull() {
    // Arrange
    var list = Arrays.asList(1, null, 3);
    var expected = new Integer[]{1, null, 3};
    var listConverter = new ListConverter();

    // Act
    var actual = listConverter.toArray(list);

    // Assert
    assertEquals(expected, actual);
}
```


MyStack<T> mit ArrayList

Implementiere eine eigene **generische** Klasse MyStack<T>, die **intern** eine **ArrayList** verwendet und folgende Methoden besitzt:

// Fügt das element ans Ende des Stacks hinzu.

```
public void push(T element)
```

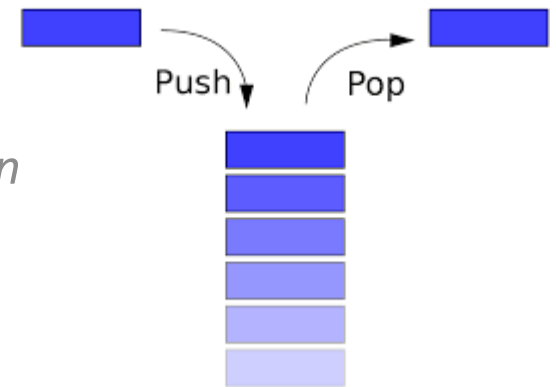
// Entfernt das zuletzt gepushte element aus dem Stack und gibt es zurück.

// Wenn kein Element vorhanden ist, wird eine NoSuchElementException geworfen

```
public T pop() throws NoSuchElementException
```

// true, wenn keine Elemente enthalten sind, sonst false .

```
public boolean isEmpty()
```



Schreibe einen Unit Test für die Klasse MyStack<T>.

Hinweis:

Die Klassen Stack, Deque und LinkedList sind nicht erlaubt für diese Aufgabe.

MyStack<T> mit ArrayList: Lösung

```
public class MyStack<T> {  
  
    private final List<T> elements = new ArrayList<>();  
  
    public void push(T element) {  
        elements.addLast(element); // oder einfach add  
    }  
  
    public T pop() throws NoSuchElementException {  
        return elements.removeLast(); // wirft bereits NoSuchElementException  
    }  
  
    public boolean isEmpty() {  
        return elements.isEmpty();  
    }  
}
```

```
class MyStackTest {  
  
    @Test  
    void popNonEmptyStack() {  
        // Arrange  
        MyStack<Integer> stack = new MyStack<>();  
        stack.push(1);  
        stack.push(2);  
  
        // Act / Assert  
        assertEquals(2, stack.pop());  
        assertEquals(1, stack.pop());  
    }  
  
    @Test  
    void popEmptyStackThrowsException() {  
        // Arrange  
        MyStack<Integer> stack = new MyStack<>();  
  
        // Act & Assert  
        assertThrows(NoSuchElementException.class, stack::pop);  
    }  
  
    @Test  
    void isEmpty() {  
        // Arrange  
        MyStack<Integer> stack = new MyStack<>();  
  
        // Act & Assert  
        assertTrue(stack.isEmpty());  
        stack.push(1);  
        assertFalse(stack.isEmpty());  
        stack.pop();  
        assertTrue(stack.isEmpty());  
    }  
}
```

MyStack<T> mit Array

Schreibe die **generische** Klasse **MyStack<T>** um (aka Refactoring), sodass sie **intern** ein **Object-Array** (Object[]) verwendet, um die Elemente zu speichern.

Falls das interne Array voll ist, soll ein neues Array mit der doppelten Größe angelegt werden.
(ohne Optimierung: das Reduzieren des Arrays muss nicht implementiert werden).

Verwende denselben (deinen) Unit Test von der vorherigen Aufgabe.

Die Klasse hat weiterhin folgende Methoden:

// Fügt das element ans Ende des Stacks hinzu.

```
public void push(T element)
```

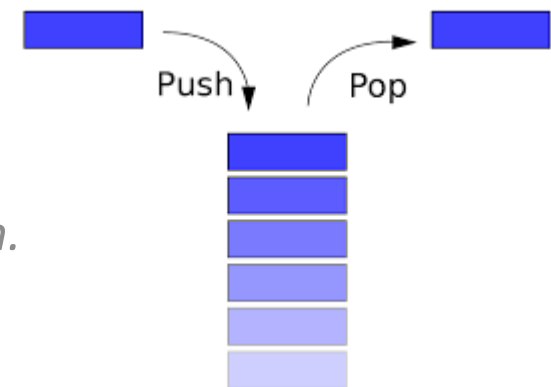
// Entfernt das zuletzt gepushte element aus dem Stack und gibt es zurück.

// Wenn kein Element vorhanden ist, wird eine NoSuchElementException geworfen.

```
public T pop() throws NoSuchElementException
```

// true, wenn keine Elemente enthalten sind, sonst false .

```
public boolean isEmpty()
```



MyStack<T> mit Array: Lösung

```
public class MyStack<T> {  
  
    private static final int INITIAL_CAPACITY = 10;  
    private Object[] elements;  
    private int size = 0;  
  
    public MyStack() {  
        this(INITIAL_CAPACITY);  
    }  
  
    public MyStack(int initialCapacity) {  
        elements = new Object[initialCapacity];  
    }  
  
    public void push(T value) {  
        ensureCapacity();  
        elements[size++] = value;  
    }  
  
    @SuppressWarnings("unchecked")  
    public T pop() {  
        if (isEmpty()) {  
            throw new NoSuchElementException();  
        }  
        T value = (T) elements[--size];  
        elements[size] = null;  
        return value;  
    }  
  
    public boolean isEmpty() {  
        return size == 0;  
    }  
  
    private void ensureCapacity() {  
        if (!hasCapacity()) {  
            enlargeArray();  
        }  
    }  
  
    private boolean hasCapacity() {  
        return elements.length < size;  
    }  
  
    private void enlargeArray() {  
        Object[] newArray;  
        if (elements.length > 0) {  
            newArray = new Object[elements.length * 2];  
        } else {  
            newArray = new Object[1];  
        }  
        System.arraycopy(elements, 0, newArray, 0, elements.length);  
        elements = newArray;  
    }  
}
```

```
class MyStackTest {  
  
    @Test  
    void popNonEmptyStack() {  
        // Arrange  
        MyStack<Integer> stack = new MyStack<>();  
        stack.push(1);  
        stack.push(2);  
  
        // Act / Assert  
        assertEquals(2, stack.pop());  
        assertEquals(1, stack.pop());  
    }  
  
    @Test  
    void popEmptyStackThrowsException() {  
        // Arrange  
        MyStack<Integer> stack = new MyStack<>();  
  
        // Act & Assert  
        assertThrows(NoSuchElementException.class, stack::pop);  
    }  
  
    @Test  
    void isEmpty() {  
        // Arrange  
        MyStack<Integer> stack = new MyStack<>();  
  
        // Act & Assert  
        assertTrue(stack.isEmpty());  
        stack.push(1);  
        assertFalse(stack.isEmpty());  
        stack.pop();  
        assertTrue(stack.isEmpty());  
    }  
}
```

```
// Es ist sinnvoll, die Capacity noch zu testen, also prüfen, ob das interne Array korrekt  
// vergrößert wird, sobald es voll ist.  
// Allerdings lässt sich das mit der aktuellen API von MyStack<T> nur indirekt prüfen:  
@Test  
void enlargeCapacity() {  
    // Arrange  
    int initialCapacity = 0;  
    MyStack<Integer> stack = new MyStack<>(initialCapacity);  
  
    // Act & Assert  
    stack.push(1);  
    assertFalse(stack.isEmpty());  
    stack.push(2);  
    assertFalse(stack.isEmpty());  
    stack.pop();  
    stack.pop();  
    assertTrue(stack.isEmpty());  
}
```

Longest String

Finde in einer Collection den längsten String und gebe ihn zurück.
Falls kein String gefunden wird, gebe einen leeren String "" zurück.

Beispiel:

```
Collection<String> words = List.of("Apple", "Orange", "Strawberry", "Banana");  
String longestString = LongestString.findLongestString(words); // Strawberry
```

Überlege dir, welcher Edge-Case hier wahrscheinlich auftreten könnte.
Schreibe einen Unit Test für deine Lösung.

Longest String: Lösung

```
static String findLongestString(Collection<String> words) {  
    String longestString = "";  
    for (var word : words) {  
        if (word.length() > longestString.length()) {  
            longestString = word;  
        }  
    }  
    return longestString;  
}
```

// Alternativen

```
static String findLongestString(Collection<String> words) {  
    return words.stream().max(String::compareTo).orElse("");  
    return words.stream().max((w1, w2) -> w1.length() - w2.length()).orElse("");  
    return words.stream().max(Comparator.comparingInt(String::length)).orElse("");  
}
```

Longest String – Unit Test: Lösung

```
class LongestStringTest {  
  
    @Test  
    void testFindLongestString() {  
        // Arrange  
        var strings = List.of("Apple", "Orange", "Banana", "Strawberry");  
        // Act  
        String longestString = LongestString.findLongestStringLoop(strings);  
        // Assert  
        assertEquals("Strawberry", longestString);  
    }  
  
    @Test  
    void testFindLongestStringWithEmptyCollection() {  
        // Arrange  
        List<String> strings = List.of();  
        // Act  
        String longestString = LongestString.findLongestStringLoop(strings);  
        // Assert  
        assertEquals("", longestString);  
    }  
}
```

```
@Test  
void testFindLongestStringWithSingleElement() {  
    // Arrange  
    var strings = List.of("Apple");  
    // Act  
    String longestString = LongestString.findLongestStringLoop(strings);  
    // Assert  
    assertEquals("Apple", longestString);  
}  
  
@Test  
void testFindLongestStringWithEqualLengthStrings() {  
    // Arrange  
    var strings = List.of("Cat", "Dog", "Fox"); // Edge Case  
    // Act  
    String longestString = LongestString.findLongestStringLoop(strings);  
    // Assert  
    assertEquals("Cat", longestString);  
}  
  
@Test  
void testFindLongestStringNull() {  
    // Arrange  
    var strings = Arrays.asList("Cat", null, "Horse");  
  
    // Assert  
    assertThrows(NullPointerException.class, () -> LongestString.findLongestStringLoop(strings));  
}  
}
```

Linear Search

Implementiere eine Methode, die eine **Collection** von ganzen Zahlen sowie eine gesuchte Zahl entgegennimmt.

Die Methode gibt den **Index** der gesuchten Zahl in der **Collection** zurück, falls die Zahl vorhanden ist, sonst -1.

Bei mehrfachen Vorkommen der gesuchten Zahl, soll der **Index** der zuerst gefundenen Zahl zurückgegeben werden.

Tipp: Es könnte hilfreich sein, die empfangene Collection in eine Liste umzuwandeln.

Beispiel:

```
Collection<Integer> numbers = List.of(1, 2, 3, 4, 5);  
int index = LinearSearch.findIndexOfNumber(numbers, 3); // 2
```

Schreibe einen Unit Test für deine Lösung.

Linear Search: Lösung

```
static int findIndexOfNumber(Collection<Integer> numbers, int numberToSearchFor) {  
    return new ArrayList(numbers).indexOf(numberToSearchFor);  
}
```

```
var numbersAsList = new ArrayList<>(numbers);  
for (int index = 0; index < numbersAsList.size(); index++) {  
    if (numbersAsList.get(index) == numberToSearchFor) {  
        return index;  
    }  
}  
return -1;  
}
```

```
static int findIndexOfNumberStream(Collection<Integer> numbers, int numberToSearchFor) {  
    var numbersAsList = new ArrayList<>(numbers);  
    return IntStream.range(0, numbersAsList.size())  
        .filter(index -> numbersAsList.get(index) == numberToSearchFor)  
        .findFirst()  
        .orElse(-1);  
}
```

Linear Search – Unit Test: Lösung

```
class LinearSearchTest {
```

```
    @Test
    void testFindIndexOfNumberPresent() {
        // Arrange
        var numbers = List.of(1, 2, 3, 4, 5);
        // Act
        int index = LinearSearch.findIndexOfNumber(numbers, 3);
        // Assert
        assertEquals(2, index);
    }
```

```
    @Test
    void testFindIndexOfNumberNotPresent() {
        // Arrange
        var numbers = List.of(1, 2, 3, 4, 5);
        // Act
        int index = LinearSearch.findIndexOfNumber(numbers, 6);
        // Assert
        assertEquals(-1, index);
    }
```

```
    @Test
    void testFindIndexOfNumberEmptyCollection() {
        // Arrange
        List<Integer> numbers = List.of();
        // Act
        int index = LinearSearch.findIndexOfNumber(numbers, 1);
        // Assert
        assertEquals(-1, index);
    }
```

```
    @Test
    void testFindIndexOfNumberMultipleOccurrences() {
        // Arrange
        var numbers = List.of(1, 3, 3, 4, 5);
        // Act
        int index = LinearSearch.findIndexOfNumber(numbers, 3);
        // Assert
        assertEquals(1, index);
    }
```

```
    @Test
    void testFindIndexOfNumberNull() {
        // Arrange
        var numbers = Arrays.asList(1, null, 3);

        // Act
        int index = LinearSearch.findIndexOfNumber(numbers, 3);

        // Assert
        assertEquals(2, index);
    }
}
```

Find MinMax

Implementiere eine Methode, die eine Collection von Ganzzahlen entgegennimmt und ein **record** MinMax mit zwei Attributen zurückgibt: dem kleinsten und dem größten **Wert** in der Eingabe-Collection.

Falls nur ein Element vorhanden ist, ist dieses Element zugleich Minimum und Maximum.

Wenn keine gültigen Werte gefunden werden, wird eine **NoSuchElementException** geworfen.

Beispiel:

```
Collection<Integer> integers = List.of(1, 2, 3, 4, 5, 0);  
MinMax minMax = MinMaxUtil.findMinMax(integers); // MinMax[min: 0, max: 5]
```

Schreibe einen Unit Test für deine Lösung.

Find MinMax: Lösung

```
record MinMax(int min, int max) {}
```

```
static MinMax findMinMax(Collection<Integer> numbers){
    int min = numbers.stream().mapToInt(Integer::intValue).min().getAsInt(); // wirft eine NoSuchElementException, falls Element nicht vorhanden
    int max = numbers.stream().mapToInt(Integer::intValue).max().getAsInt();

    // int min = numbers.stream().min(Integer::compare).orElseThrow(NoSuchElementException::new);
    // int max = numbers.stream().max(Integer::compare).orElseThrow(NoSuchElementException::new);

    // int min = numbers.stream().mapToInt(i->i).min().orElseThrow(NoSuchElementException::new);
    // int max = numbers.stream().mapToInt(i->i).max().orElseThrow(NoSuchElementException::new);

    return new MinMax(min, max);
}

static MinMax findMinMax(List<Integer> numbers) {
    if(numbers.isEmpty()) {
        throw new NoSuchElementException("List is empty");
    }

    int min = Integer.MAX_VALUE;
    int max = Integer.MIN_VALUE;
    for (int number : numbers) {
        if (number < min) {
            min = number;
        }
        if (number > max) {
            max = number;
        }
    }
    return new MinMax(min, max);
}
```

Find MinMax: Lösung

```
class MinMaxUtilTest {
```

```
    @Test
```

```
    void testFindMinMax() {
```

```
        // Arrange
```

```
        List<Integer> integers = List.of(1, 2, 3, 4, 5);
```

```
        // Act
```

```
        MinMax actual = MinMaxUtil.findMinMax(integers);
```

```
        // Assert
```

```
        assertEquals(1, actual.min());
```

```
        assertEquals(5, actual.max());
```

```
        // Oder alternativ (da MinMax ein record ist):
```

```
        assertEquals(new MinMax(1, 5), actual);
```

```
    }
```

```
    @Test
```

```
    void testFindMinMaxWithNegativeNumbers() {
```

```
        // Arrange
```

```
        List<Integer> integers = List.of(-1, -2, -3, -4, -5);
```

```
        // Act
```

```
        MinMax actual = MinMaxUtil.findMinMax(integers);
```

```
        // Assert
```

```
        assertEquals(-5, actual.min());
```

```
        assertEquals(-1, actual.max());
```

```
    }
```

```
    @Test
```

```
    void testFindMinMaxWithDuplicates() {
```

```
        // Arrange
```

```
        List<Integer> integers = List.of(1, 2, 3, 5, 1, 3, 4, 5);
```

```
        // Act
```

```
        MinMax actual = MinMaxUtil.findMinMax(integers);
```

```
        // Assert
```

```
        assertEquals(1, actual.min());
```

```
        assertEquals(5, actual.max());
```

```
    }
```

```
    @Test
```

```
    void testFindMinMaxOneNumberOnly() {
```

```
        // Arrange
```

```
        List<Integer> integers = List.of(1);
```

```
        // Act
```

```
        MinMax actual = MinMaxUtil.findMinMax(integers);
```

```
        // Assert
```

```
        assertEquals(1, actual.min());
```

```
        assertEquals(1, actual.max());
```

```
    }
```

```
    @Test
```

```
    void testFindMinMaxAllIdenticalNumbers() {
```

```
        // Arrange
```

```
        List<Integer> integers = List.of(1, 1, 1);
```

```
        // Act
```

```
        MinMax actual = MinMaxUtil.findMinMax(integers);
```

```
        // Assert
```

```
        assertEquals(1, actual.min());
```

```
        assertEquals(1, actual.max());
```

```
    }
```

```
    @Test
```

```
    void emptyList() {
```

```
        assertThrows(NoSuchElementException.class, () -> MinMaxUtil.findMinMax(List.of()));
```

```
    }
```

```
}
```